Brainfuck Interpretation and Optimization

Max Krummenacher

1. Brainfuck

Brainfuck is an minimalist language operating on a tape of zero initialised unsigned bytes through a pointer to that tape. The tape is bounded on the left and unbounded to the right, although implementations usually use an upperbound for practical purposes.

The operations of brainfuck are shown in Table 1.

Instruction	Effect	C equivalent
+	increase the value pointed at by one	`(*t_ptr)++;`
-	decrease the value pointed at by one	`(*t_ptr)--;`
>	increase the tape pointer by one	`t_ptr++;`
<	decrease the tape pointer by one	`t_ptr--;`
[	if the value pointed at is zero goto the instruction after the matching]	`while (*t_ptr != 0) {`
]	if the value pointed at is non zero goto the instruction after the matching [	`}`
.	output the byte at the tape pointer	`putchar(*t_ptr);`
,	get one byte of input and write it to the tape	`*t_ptr = getchar();`

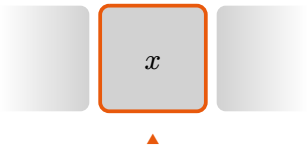
Table 1: Brainfuck Instructions

1.1. Comments

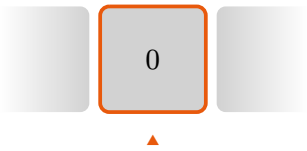
Any character which does not match any of the described operations is considered a comment. Additionally, if a program starts with `[` anything until the matching `]` can be considered a comment. This second type of comment can use any character, including operations, with the only limmitation that `[` and `]` need to be matched. Since the tape is zero initialized any code written here is skipped.

1.2. Examples

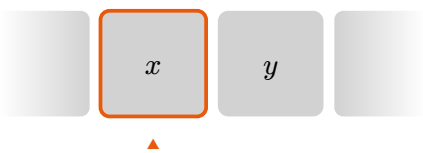
1.2.1. Setting a value equal to zero



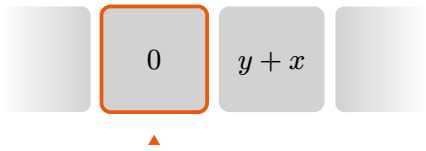

```
[-]
```



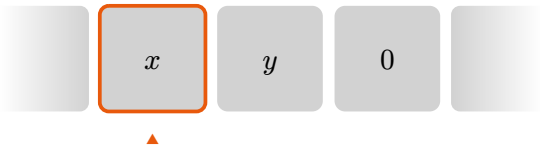
1.2.2. Addition



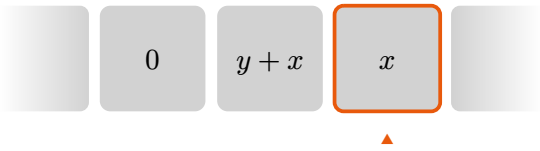
```
[>+<-]
```



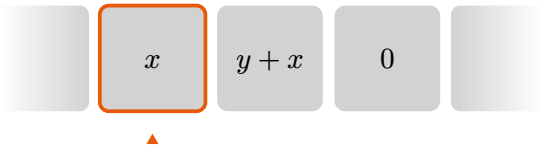
1.2.3. Addition Preserving original value



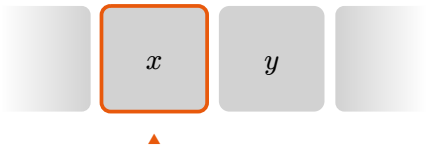

```
[>+>+<<-]
>>
```



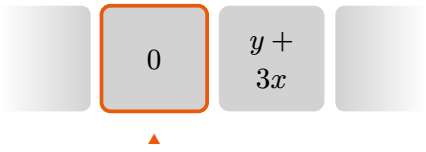

```
[<<+>>-]
<<
```



1.2.4. Multiplication by Constant




```
[>+++<-]
```



2. Intermediate representation

The brainfuck code is first translated into an intermediate representation (IR). The instructions can operate on an immediate offset to the tape pointer. These offsets are indicated with ^.

The operations of the IR are shown in Table 2.

Operation	Description	C equivalent
`ADD ^off imm`	add `imm` to the value at tape pointer + `off`	`*(t_ptr + off) += imm;`
`MOV imm`	move the tape pointer by `imm`	`t_ptr += imm;`
`LSTART`	jump forward to the instruction after the matching `LEND` if the value at the tape pointer is zero	`while (*t_ptr != 0) {`
`LEND`	jump backward to the instruction after the matching `LSTART` if the value at the tape pointer is non-zero	`}`
`PCH ^off`	output the value at tape pointer + `off`	`putchar(*(t_ptr + off));`
`GCH ^off`	get a byte of input and write it to tape pointer + `off`	`*(t_ptr + off) = getchar();`
`MAD ^dst ^src imm`	multiply the value at tape pointer + `src` by `imm` and add the result to the value at tape pointer + `dst`	`*(t_ptr + dst) = *(t_ptr + src) * imm;`
`CLR ^off`	set the value at tape pointer + `off` to zero	`*(t_ptr + off) = 0;`
`NOP`	do nothing	-

Table 2: IR-Instructions

3. Canonicalization of Basic Blocks

A basic block is a segment of IR which does not contain any of `LSTART`, `LEND`. The `MOV` operations are collected and put at the end of a basic block and the offsets to the arithmetic and io operations are updated accordingly.

Since the order of some instructions does not matter we can sort them.

Sorting happens by insertion sort and runs until we en

```
ADD ^0  1
MAD ^0 ^1  4
```

is the same as

```
MAD ^0 ^1  4
ADD ^0  1
```

since it is just the transformation form `[x, y]` to `[x + 1 + 4*y, y] = [x + 4*y + 1, y]` But

```
ADD ^0  1
MAD ^1 ^0  4
```

`{..., x, y] -> [x+1, y + 4 * (x+1)]` and

```
MAD ^1 ^0  4
ADD ^0  1
```

`[x, y] -> [x+1, y + 4*x]` lead to different outcomes

Additionally the arithmetic operations are sorted by the offset to which they are applied, such that arithmetic operations on the same value can be combined.

4. Underflow and Overflow Handling

Underflows and overflows are handled by the operating system. The tape region is a multiple of the page size of the system, which allows me to allocate a page with no permission below and above the valid region. Out of bounds access is then handeled by catching the segfault and is determined to be an underflow or overflow by which of the guard regions is accessed.

4.1. Limmitations

If the memory access by brainfuck is outside the bounds by more than a region an unknown error is returned, as I cannot be sure if the access was meant to access the tape.

The validity of a tape address is only checked on access to a value. The tape pointer is allowed to be invalid if it is never accessed. The programm `<> +` does not report an underflow even though after `<` it points outside the tape region.

5. Translation to Aarch64 Assembly

The assembly for the one brainfuck file is translated to a single function: `void bf_assembled(void *tape_base, int *putchar(int), int *getchar(void));`

One register holds the tape pointer (`xtape`) throughout the whole program.

`xtape` is chosen to be x19, since it is a callee preserved register.

To call `getchar` and `putchar` the function pointers are passed to the function. Simmilar to `xtape` callee preserved registers (`xgetc` and `xputc`) are used for them.

5.1. Prelude

```
; allocate frame, save {fp, lr} at [sp, 0]
stp fp, lr, [sp, -48]!

; point fp to {fp, lr} record
mov fp, sp

; preserve callee preserved registers
stp xtape, xunused, [sp, 16]
stp xputc, xgetc, [sp, 32]

; move arguments into correct registers
mov xtape, x0
mov xputc, x1
mov xgetc, x2
```

5.2. Postlude

```
; restore callee preserved registers
ldp xtape, xunused, [sp, 16]
ldp xputc, xgetc, [sp, 32]

; restore {fp, lr} and deallocate
ldp fp, lr, [sp], 48
ret
```

the following sections are stale

5.3. (MOV, n)

If n > 0

```
add xtape, xtape, n
```

If n < 0

```
sub xtape, xtape, -n
```

5.4. (ADD, n, o)

If n != 0

```
ldurb w0, [xtape, o]
add w1, w0, (n % 256)
sturb w1, [xtape, o]
```

5.5. (LSTART, a)

```
ldrb w0, [xtape]
cbz w0, offset(a)
```

5.6. (LEND, a)

```
ldrb w0, [xtape]
cbnz w0, offset(a)
```

5.7. (PCH, n)

```
ldrb w0, [xtape]
blr xputc
```

repeated n times

5.8. (GCH, n)

```
blr xgetc
strb w0, [xtape]
```

repeated n times